
Written Values Are Read Back: The Chain of Thought as Addressable Memory

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 When a language model writes an intermediate value during multi-step reason-
2 ing, the written token could carry the computation forward or describe work done
3 elsewhere in the network. Prior work has shown, on a fine-tuned model and on
4 multiplication and dynamic-programming tasks, that intervening on such tokens
5 changes the answer. We test the question on unmodified pretrained models across
6 four families and two task formats, and localize the mechanism. Overwriting the
7 internal state at a written value’s token with the state a different number would
8 have produced, while leaving the visible text unchanged, makes the final answer
9 track the never-written value in 74 to 100 percent of cases (six models, arithmetic
10 and narrative tasks), while a size-matched random perturbation moves the answer
11 in under 3 percent. Blocking attention from later positions to that one token drops
12 the answer’s dependence on it to near zero on four models in three families, while
13 blocking a neighboring token, a random prompt token, or an earlier written value
14 leaves it unchanged; the model continues to produce fluent, parseable answers, so
15 the intervention removes access to the value rather than disabling generation. Two
16 values planted at two positions compose into their sum, which appears nowhere
17 in the input. Models consult this stored value by default, and which prompt evi-
18 dence leads them to recompute it instead differs by model. On serially dependent
19 state that a model cannot cheaply regenerate, the written trace is the memory the
20 computation reads, which bears directly on whether reading a model’s chain of
21 thought reveals what it computed.

22 1 Introduction

23 Oversight of language-model reasoning often assumes that the text a model writes reflects the com-
24 putation it performs. Faithfulness studies have shown that stated reasoning and internal computation
25 can diverge [Turpin et al., 2023, Lanham et al., 2023]. This leaves a specific mechanistic question
26 open: when a model writes “after step 5 the value is 452” and later uses 452, does the written token
27 cause the later use, or does the computation happen elsewhere and the token record it?

28 Causal interventions on written reasoning tokens are not new. Shih et al. [2026] patch internal repre-
29 sentations of scratchpad state on a fine-tuned model and report downstream following of 0.80 to 0.91,
30 and Zhu et al. [2025] intervene on value-storing tokens in multiplication and dynamic-programming
31 tasks. We extend this in four directions. First, the effect holds on *unmodified* pretrained models
32 across four families (Qwen, Phi, OLMo, DeepSeek-R1-Distill) and two task formats. Second, we
33 localize the read to attention directed at the value’s own token, with matched-token controls and a
34 fluency check that rules out simply disabling generation. Third, planted values compose: two posi-
35 tions act as independent stores. Fourth, we map when a model recomputes a value instead of reading
36 it, and find the answer is model-specific rather than a single scale trend.

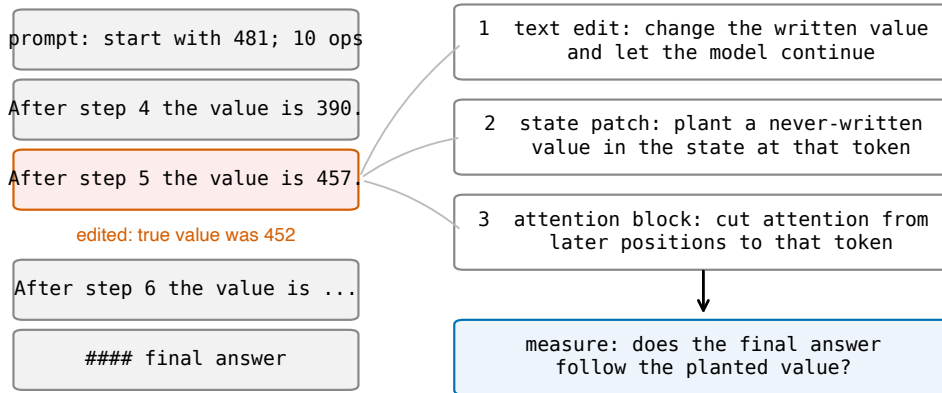


Figure 1: The edit-and-continue protocol. A correct worked solution is truncated after one edited value line, and the model continues. Three interventions act on the edited value: changing the visible token, overwriting the internal state at that token with the state a different value would produce, and blocking attention from later positions to that token. In each case we measure whether the final answer follows the planted value.

37 The central result is that a written intermediate value is held at its token position as state that later
 38 computation reads back through attention, and that this stored value, not the visible text and not the
 39 value the problem implies, determines the answer. We establish it with three interventions on one
 40 protocol (Figure 1): edit the visible text, overwrite the internal state, and block the attention that
 41 reads the token.

42 2 Setup

43 **Tasks.** We use two families of problems for which every intermediate value is known by con-
 44 struction, so the effect of any edit is computable exactly. Arithmetic chains start from a three-digit
 45 number and apply d signed two-digit additions and subtractions. Narrative state-tracking problems
 46 describe a character whose count of some item changes through varied events, with varied names,
 47 items, verbs, and sentence forms. In both, the model writes one line per step giving the running
 48 value.

49 **Protocol.** We take a correct worked solution, change the value at one step j by a small amount,
 50 truncate immediately after the edited line, and let the model continue. A no-edit truncation measures
 51 the sampling noise floor, which is 0.00 throughout. Answers are parsed from a fixed final-line
 52 format, and unparseable outputs are counted separately.

53 **Interventions.** The *text edit* changes only the visible token. The *state patch* leaves the text intact
 54 and overwrites the residual stream at the value’s token position, across a band of middle layers, with
 55 activations captured from a run in which a different value was written there. The *attention block*
 56 adds a mask so that no position after the edit can attend to the value’s token.

57 **Models and reporting.** White-box experiments run on Qwen3-4B, Qwen2.5-7B-Instruct, Phi-
 58 3-medium, OLMo-2-7B-Instruct, and DeepSeek-R1-Distill-Qwen 7B and 14B. Frontier models
 59 (DeepSeek V3.2, Claude Sonnet 4.5, Llama 3.3 70B) are tested behaviorally through assistant pre-
 60 fill. Generation uses temperature 0.6 and top- p 0.95, with five samples per item where sampling is
 61 used; the noise floor is measured under the same settings. Reported intervals are 95 percent boot-
 62 strap intervals resampling over items. We report intervals rather than significance tests and apply no
 63 multiple-comparison correction; the effects that carry the argument are separated from their controls
 64 by margins far larger than the intervals. Sample sizes are 46 to 600 per condition. Model revisions,
 65 prompts, exact layer bands, and per-run configurations are in the appendix, and code and data will
 66 be released.

Table 1: State patch on arithmetic chains at ten steps. Rates are conditioned on items where the text edit was followed (that rate is the first column), so cross-model comparison of the later columns should account for it. Brackets are 95 percent bootstrap intervals. The two reasoning-distilled models re-solve the problem after their reasoning phase, which raises their random-control reversion; that re-derivation can only produce the correct answer and so does not affect the planted-value column.

model	text edit followed	restore correct	plant new value	random control	<i>n</i>
Qwen3-4B	0.98 [0.95, 1.00]	1.00 [1.00, 1.00]	1.00 [0.99, 1.00]	0.00 [0.00, 0.00]	106
Qwen2.5-7B-Instruct	0.85 [0.80, 0.90]	0.97 [0.92, 1.00]	0.74 [0.67, 0.81]	0.00 [0.00, 0.01]	93
Phi-3-medium	0.94 [0.90, 0.98]	0.98 [0.95, 1.00]	0.94 [0.91, 0.97]	0.00 [0.00, 0.00]	102
OLMo-2-7B	0.85 [0.79, 0.91]	0.88 [0.82, 0.94]	0.93 [0.88, 0.97]	0.00 [0.00, 0.00]	100
R1-distill-7B	0.60 [0.53, 0.66]	1.00 [0.99, 1.00]	0.76 [0.68, 0.84]	0.30 [0.22, 0.39]	65
R1-distill-14B	0.44 [0.36, 0.52]	0.99 [0.98, 1.00]	0.70 [0.61, 0.80]	0.21 [0.13, 0.29]	46

67 3 The state at a written token holds the value

68 Editing the visible text alone changes the final answer on most items. The state patch decomposes
69 this effect (Table 1). Restoring the correct internal state while the visible text still shows a corrupted
70 value returns the answer to correct in 0.88 to 1.00 of cases. Planting a third value that appears
71 nowhere in the text makes the answer track that value in 0.70 to 1.00 of cases. The size-matched
72 random control is at or below 0.01 for the instruction-tuned models. The effect persists with chain
73 length: on Qwen3-4B the planted value is followed at 0.98, 0.97, and 0.95 for chains of 5, 20, and
74 40 steps.

75 **The reasoning-distilled models.** R1-distill-7B and 14B show an elevated random-control rever-
76 sion (0.30 and 0.21) because they re-solve the problem after their reasoning phase. In a logged
77 sample of the reverting cases, the continuation re-derives from the problem’s starting value rather
78 than correcting the value in place. Re-solving can only produce the correct answer, so it inflates
79 the random control without touching the planted-value column, where the answer is a value the
80 model never wrote and the problem never implies. R1-distill-1.5B is excluded: at 12 usable items
81 its competence is too low to interpret.

82 **Not the arithmetic template.** On the narrative task the pattern replicates. Planting a never-written
83 count is followed at 0.83 [0.75, 0.90] on Qwen3-4B and 0.86 [0.79, 0.92] on Phi-3-medium, restor-
84 ing the correct state returns the answer at 0.89 and 0.94, and the random control is 0.00 in both
85 ($n = 86$ and 89 conditioned items). The stored value is a property of written state, not of one
86 surface form.

87 **Two positions act as independent stores.** In a task with two counters summed at the end, we
88 plant a never-written value at each of the two final positions. Each single-position patch yields the
89 corresponding mixed sum (0.94 and 0.93 on Qwen2.5-7B, 0.89 and 0.93 on Qwen3-4B). Planting
90 both yields their joint sum at 0.90 and 0.88, at or above the product of the single-position rates (0.89
91 and 0.83). The size-matched random patch leaves the clean answer in place (0.96 and 0.95), and
92 answers matching only one of the two planted values occur at 0.01. The model reports a number
93 that exists nowhere in its input, assembled from two independently set stores.

94 4 The value is read back through attention to its token

95 Blocking attention from all positions after the edit to the edited value’s token collapses the answer’s
96 dependence on that value (Table 2). The same block applied to the adjacent words on the same line,
97 to a random operand in the prompt, or to an earlier step’s written value leaves the dependence at
98 its no-block level. Blocking the operand that the next step needs breaks the arithmetic itself, giving
99 answers that match neither the edit nor the correct value. The collapse holds on four models across
100 three families and on both task formats (Figure 2).

101 **The block removes the value, not the ability to generate.** Under the value-token block the
102 unparseable-output rate stays at 0.000 (0.040 on OLMo-2-7B, equal to its no-block rate). The

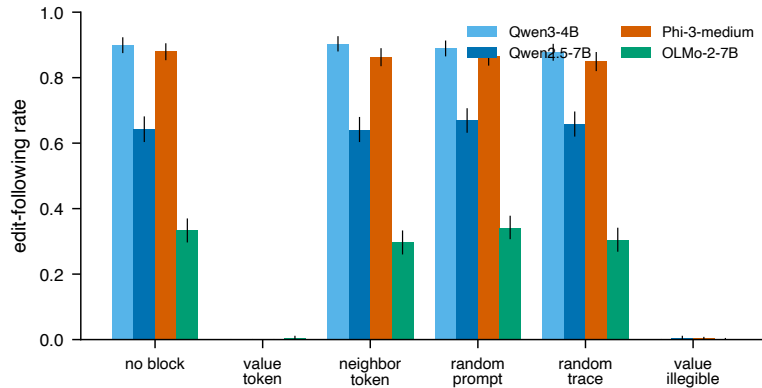


Figure 2: Rate at which the answer follows the edited value when attention to a chosen token is blocked. Blocking the value’s own token collapses following on every model; blocking a neighboring token, a random prompt token, or an earlier written value leaves it at the no-block level. Error bars are 95 percent bootstrap intervals.

Table 2: Attention block on arithmetic chains: rate at which the answer follows the edited value under each blocked target. Value blocks the edited value’s token; neighbor, random prompt token, and earlier written value are matched controls. OLMo-2-7B has a low no-block rate in this format but the same contrast. The natural-language task shows the same collapse (Qwen3-4B 0.730 to 0.007; Phi-3-medium 0.720 to 0.003).

model	no block	value token	neighbor	random token	earlier value
Qwen3-4B	0.900	0.000	0.903	0.778	—
Qwen2.5-7B-Instruct	0.643	0.000	0.640	0.670	0.658
Phi-3-medium	0.880	0.000	0.863	0.865	0.850
OLMo-2-7B	0.333	0.005	0.297	0.342	0.305

103 model produces a fluent, parseable number; that number simply matches neither the edited value
 104 nor the correct one, because the model computes forward from a value it can no longer retrieve. The
 105 intervention removes access to the specific value rather than disabling generation near that position.

106 **Read-back is the preferred path, not the only one.** Replacing the value’s characters with dots,
 107 without touching attention, also removes following, but differently: the Qwen models reconstruct
 108 the value from the previous line and return the correct answer 60 to 73 percent of the time, while
 109 Phi-3-medium does not recover (0.00). When the token is present the model reads it; when it is
 110 unreadable, some models can rebuild it and by default do not.

111 **What this does and does not establish.** The block masks the value’s token at every layer, and the
 112 state patch overwrites a band of middle layers. These interventions show that the value at the written
 113 token is causally load-bearing and that attention to that token is the conduit. They do not yet localize
 114 the read to a specific layer range. A layer-resolved sweep, patching and masking single layers and
 115 thirds of the network, is the natural next step and would separate a compact retrieval circuit from
 116 a distributed one; we report it as the primary open item. The controls above rule out the cruder
 117 alternatives available without it: the effect is specific to the value token rather than any trace token,
 118 and the model remains fluent, so “following drops to zero” is not merely broken generation.

119 5 When the written value is checked

120 Models follow edited values by default. Whether prompt evidence leads a model to recompute a
 121 value instead depends on the model and on the kind of evidence, and we did not find it to be a single
 122 trend in model capability.

On DeepSeek V3.2, editing a value the prompt only implies is followed at 0.93, while editing a value the prompt generatively defines (the prompt states the two numbers whose sum becomes the running value) is reverted to the correct answer at 0.43. The check is targeted: with the defining sentence present but the edit two steps away from the defined value, following returns to 0.96, equal to the implied-value case. Claude Sonnet 4.5 checks a wider range of evidence, reverting most when a stated checkpoint is the value’s only source. Llama 3.3 70B follows edits at 0.97 whether or not the value is prompt-defined, despite being able to solve the same chains from scratch, so capacity to recompute does not imply checking. Qwen models at 4B and 7B never revert.

These conditions come from separate experiments with different trace lengths and item sets; each within-experiment contrast is controlled, and we do not claim a single ordered scale across them. The behavioral edit-following rates on frontier models (R1-distill-7B 0.42, V3.2 0.78, Sonnet 4.5 0.97 on prompt-underivable chains) do not extend monotonically to Llama 3.3 70B at 0.97, so we do not claim that trace-dependence increases with capability in general. The mechanistic results of Sections 3 and 4 are established on the white-box models; the frontier results are behavioral.

6 Scope

The mechanism concerns state a model cannot cheaply regenerate from the prompt. The same edit-and-continue test on natural benchmark problems bounds it: on GSM8K worked solutions, editing an intermediate value changes the answer at 0.10 on V3.2 and 0.87 on Qwen3-4B, because a benchmark intermediate is often one operation from numbers still in the prompt, and whether a model rereads it or recomputes it depends on the model’s own capacity. Written values act as memory for state that recomputation cannot cheaply reconstruct, and much of benchmark arithmetic does not have that character.

7 Related work

Our interventions build on causal tracing and activation patching [Meng et al., 2022] and causal abstraction [Geiger et al., 2021]. Heimersheim and Nanda [2024] caution that broad patches conflate “this component matters” with deleting information at a position, which motivates the layer sweep we identify as the open item. The closest empirical work intervenes on written reasoning tokens: Shih et al. [2026] on a fine-tuned model and Zhu et al. [2025] on multiplication and dynamic-programming tasks. We differ in testing unmodified pretrained models across four families and two task formats, localizing the read to attention to the value token with matched controls and a fluency check, showing composition, and mapping when the record is checked. Faithfulness studies established that stated reasoning and internal computation can diverge [Turpin et al., 2023, Lanham et al., 2023], and filler-token results show computation can proceed through uninformative tokens [Pfau et al., 2024]; these bound our claim to state a model cannot regenerate internally. Expressivity theory predicts that long dependency chains must live in the generated text [Merrill and Sabharwal, 2024], and Self-Notes [Lanchantin et al., 2023] studies writing as memory behaviorally.

8 Limitations

The state patch overwrites a band of middle layers and the attention block masks all layers, so the read is not yet localized to a specific depth; this is the primary revision item. The white-box causal evidence is at 4B to 14B scale, and the frontier evidence is behavioral. Tasks are arithmetic and narrative-count chains with exact ground truth; richer state such as code or multi-entity plans is untested. The checking-policy results in Section 5 draw on conditions from separate experiments and support within-experiment contrasts rather than a single ordered scale. The state patch is controlled against a size-matched random vector; a structured control that plants a valid activation for a different value would further separate value content from generic activation shape.

9 Conclusion

On two task formats and across four model families, a value written in a chain of thought is held at its token position as state that later computation reads back through attention, and that stored value

determines the answer even when it appears nowhere in the text and contradicts what the problem implies. Two positions act as independent stores. Models read this state by default, and what leads them to recompute instead is model-specific. For oversight, the consequence is scoped and concrete: on serially dependent state a model cannot cheaply regenerate, the written trace is the memory the computation uses, so reading the trace observes the computation; on state the model can regenerate, the trace and the computation can diverge.

References

- Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2106.02997.
- Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255*, 2024.
- Jack Lanchantin, Shubham Toshniwal, Jason Weston, Arthur Szlam, and Sainbayar Sukhbaatar. Learning to reason and memorize with self-notes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.00833.
- Tamera Lanham et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2202.05262.
- William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07923.
- Jacob Pfau, William Merrill, and Samuel R. Bowman. Let’s think dot by dot: Hidden computation in transformer language models. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2404.15758.
- Andy Shih et al. Do models read what they write? causal registers in scratchpad reasoning. *arXiv preprint arXiv:2606.29522*, 2026.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.04388.
- Fangwei Zhu et al. Chain-of-thought tokens are computer program variables. *arXiv preprint arXiv:2505.04955*, 2025.